



RAKSHITAH

GRC Compliance Platform

Governance · Risk · Compliance

White Paper

Business Overview · Architecture · Technical Flows

Version 1.0 | May 2026



1. Executive Summary

RAKSHITAH (रक्षितः) is an enterprise-grade Governance, Risk, and Compliance (GRC) platform developed by 9USRCraft LLP. Derived from the Sanskrit word meaning 'Protected', RAKSHITAH embodies the essence of safeguarding organizations against business uncertainties, vulnerabilities, and compliance risks.

Modern enterprises face an ever-expanding landscape of regulatory obligations, operational risks, and cybersecurity threats. Legacy approaches—spreadsheets, disconnected point tools, and manual evidence collection—fail to provide the speed, visibility, and auditability that today's governance requirements demand. RAKSHITAH solves this by unifying every GRC discipline into a single, highly configurable platform that scales from startup compliance to enterprise risk programs.

This white paper presents a comprehensive overview of RAKSHITAH's business capabilities, technical architecture, data flows, and deployment model, providing technology evaluators, CISOs, GRC practitioners, and IT architects with the depth of information required to assess the platform for enterprise adoption.

40% Reduction in Manual Compliance Effort	50% Faster Audit Readiness	10+ GRC Domains Covered	100% Business Unit Risk Visibility
--	--------------------------------------	-----------------------------------	--

2. Business Overview & Value Proposition

2.1 Problem Statement

Organizations of all sizes struggle with fragmented GRC tooling. Security teams manage risk in spreadsheets while compliance teams operate in isolated document repositories and audit teams lack real-time visibility into control status. The consequences include regulatory penalties, audit failures, security breaches, and operational inefficiency.

Key pain points RAKSHITAH addresses:

- Spreadsheet-driven risk management with no audit trail or version history
- Siloed compliance tracking disconnected from risk and governance activities
- Manual, error-prone evidence collection for audits and certifications
- Lack of a single source of truth for policies, exceptions, and controls
- Expensive commercial GRC suites out of reach for mid-market organizations
- Inability to demonstrate continuous compliance to regulators and customers

2.2 The RAKSHITAH Solution

RAKSHITAH delivers a unified, web-based GRC platform that brings together every dimension of enterprise risk management — from risk identification and scoring, through compliance framework mapping and control testing, to governance document management and audit reporting — all within a single, cohesive application.

With RAKSHITAH, enterprises can:

- Protect against Risks & Vulnerabilities — Proactively identify, assess, and mitigate risks before they impact business
- Ensure Continuous Compliance — Stay aligned with ever-changing global regulations and standards including ISO 27001, SOC 2, GDPR, HIPAA, and PCI-DSS
- Strengthen Governance — Enable transparency, accountability, and informed decision-making through centralized policy and document management
- Foster Business Resilience — Turn compliance and risk management into strategic enablers of growth

2.3 Target Audience

RAKSHITAH is designed for cross-functional GRC stakeholders across enterprise and mid-market organizations:

Role	Responsibilities in RAKSHITAH
CISOs / Security Leaders	Executive risk oversight, board reporting, risk posture visibility
GRC Managers	Framework management, compliance orchestration, program governance
Risk Analysts	Risk identification, scoring, mitigation planning, residual risk tracking
Compliance Teams	Framework mapping, control testing, evidence collection, audit readiness
Internal Auditors	Audit initiation, test execution, finding management, reopening
Control Owners	Control documentation, testing participation, mitigation assignment
IT / Security Operations	Asset management, vulnerability integration, technical risk response
Business Managers	Approval workflows, review sign-offs, exception handling
System Administrators	Platform configuration, user/role management, integrations

2.4 Key Business Value Metrics

Quantified Business Value
✓ 40% reduction in manual compliance effort through workflow automation
✓ 50% faster audit readiness with real-time evidence and dashboard tracking
✓ 100% risk visibility across all business units via the unified GRC Dashboard
✓ Enhanced regulatory confidence with automated, tamper-evident audit trails
✓ Automated report generation — SOA, Audit Reports, Compliance Summaries, and more
✓ Lower TCO vs. commercial GRC suites — flexible deployment, no per-user licensing lock-in

2.5 What Makes RAKSHITAH Unique

RAKSHITAH differentiates itself from competing GRC platforms through five core pillars:

- Flexible Deployment — Available as Cloud-hosted SaaS, On-Premises Docker/LAMP, or Hybrid deployment. Enterprises retain full data sovereignty.
- Highly Customizable — Adapt scoring methodologies, dashboards, compliance frameworks, workflows, and reference data to organizational needs without vendor dependency.
- User-Centric Design — Intuitive web UI built on modern front-end libraries (Bootstrap, DataTables, Highcharts) that reduces training effort and accelerates user adoption.
- Scalable & Modular — Start with core Risk and Compliance modules; expand to Governance, Asset Management, Assessments, Vulnerability Management, and Incident Management as maturity grows.
- Regulatory Intelligence — Built-in support for ISO 27001, SOC 2, GDPR, HIPAA, PCI-DSS, and extensible framework import for emerging regulatory requirements.
- Open Extensibility — Plugin-style extras model, REST API (v1 and v2 with Swagger/OpenAPI), and integration connectors for Jira, Azure DevOps, GitHub OAuth, SAML SSO, and more.

3. GRC Platform Modules & Features

RAKSHITAH covers ten primary GRC domains, each implemented as a dedicated module with its own UI, data model, workflow engine, and reporting surface. The modules are deeply interconnected — a failed compliance test can generate a risk, a risk can be linked to assets, and a closed risk can satisfy a compliance objective.

3.1 Risk Management

The Risk Management module is the core of RAKSHITAH, providing end-to-end lifecycle management for enterprise risks from identification through closure.

- Risk submission with full metadata: subject, reference ID, source, category, location, team, technology, owner, manager, stakeholders, and project mapping
- Six configurable scoring methodologies: Classic (Likelihood × Impact), CVSS, DREAD, OWASP, Custom formula, and Contributing Risk scoring
- Residual risk tracking and historical scoring timelines
- Mitigation planning with control linkage, effort estimation, and acceptance workflow
- Management review cycles with configurable review periods and status tracking
- Risk comments, collaboration, and attachment of supporting documentation
- Risk closure, reopening, and full audit trail of lifecycle transitions
- Dynamic saved filters, custom column views, and saved selections for power users
- Risk-to-asset and risk-to-asset-group mapping for impact scoping

3.2 Compliance Management

The Compliance module enables systematic framework-based compliance management, linking controls to test plans and audit execution.

- Framework and control library management with support for ISO 27001, SOC 2, GDPR, HIPAA, PCI-DSS, and custom frameworks
- Rich control metadata: owner, class, phase, maturity level, desired maturity, priority, status, audit dates, and mitigation percentage
- Compliance test definition: tester, stakeholders, teams, frequency, objectives, test steps, expected results, and approximate effort
- Active and past audit dashboards with real-time execution tracking
- Failed test → risk linkage: associate a failed control test directly with a new or existing risk for remediation tracking
- Compliance gap analysis reports and control-family coverage views
- Framework control mappings and cross-framework control type mappings

3.3 Audit Management

- Audit initiation workflow: select controls, assign testers, define scope
- Structured test execution with result recording, evidence attachment, and comment threads
- Timeline API for each audit with full chronological evidence trail
- Active/past audit views with filtering and drill-down capability
- Reopen capability for contested or updated audit findings
- Test results linked back to risks for closed-loop remediation

3.4 Governance & Document Management

- Centralized policy and document repository with rich-text authoring (TinyMCE WYSIWYG editor)
- Document lifecycle: create, update, version, and manage approval status
- Parent-child document and framework hierarchies for structured governance libraries
- Document exception management: create, update, approve, and audit exceptions with full approval trails
- Control-to-document linkage and asset attachment support

- Document status and exception status lifecycle tables

3.5 Asset Management

- Asset inventory with valuation, grouping, and risk linkage
- Asset group management with hierarchical relationship trees
- Risk-to-asset and risk-to-asset-group many-to-many mappings for precise impact scoping
- Assessment answers linked to assets for risk-scored asset profiles

3.6 Assessment Management

- Assessment questionnaire creation, sending, and processing workflows
- Key-based external participant access for secure, token-authenticated questionnaire completion
- Assessment answers linked to assets and asset groups for risk-correlated profiling
- Assessment scoring engine with configurable contributing impact factors

3.7 Incident & Vulnerability Management

- Incident management module for tracking and responding to security incidents
- Vulnerability management with scheduled cron-based processing and dashboard reporting
- Integration points for external vulnerability data ingestion

3.8 Reporting & Dashboards

- Unified GRC Dashboard providing a single-pane executive view of risk posture, compliance status, and audit health
- Highcharts-powered visualizations: pie charts, trend lines, bar charts by risk level, status, location, category, team, scoring method, and more
- DataTables-driven tabular reports: high-risk registers, open risks by owner, overdue reviews, control gap analysis
- Executive-level risk appetite and risk posture summaries
- User-specific display settings for personalized reporting views
- Drill-down capability from executive summary to individual risk or control detail

3.9 Administration & Platform Management

- Comprehensive user and role management with RBAC permission model
- Mail server configuration, test-mail functionality, and notification management
- Backup configuration, scheduling, and retention management
- Security settings: password policy, lockout thresholds, MFA enforcement, CSP, session timeout
- Audit trail viewer with full change log for all platform actions
- Import/export capability for risks, frameworks, and reference data
- Health check and application upgrade management
- Multi-language support (i18n) via localization packs

4. Technical Architecture

4.1 Architecture Pattern

RAKSHITAH is built on a Modular Monolith architecture — a single deployable PHP/Apache application organized into clearly bounded domain modules, sharing a common MySQL relational database and a procedural service layer in the `includes/` directory. This architectural choice prioritizes operational simplicity, deployment flexibility, and broad hosting compatibility while maintaining strong domain separation.

Architecture Style	Modular Monolith — PHP/Apache, single deployable unit, MySQL persistence
API Surface	REST-like JSON endpoints (<code>api/index.php</code> + <code>includes/api.php</code>) + Swagger/OpenAPI v2 documentation
Frontend Pattern	Server-rendered multi-page application with progressive AJAX enhancement (jQuery + Bootstrap)
Background Processing	OS Cron-driven PHP jobs for notifications, vulnerability management, assessments, and backups
Deployment	Docker Compose (PHP 8.2 + Apache + MySQL 8.0) or traditional LAMP stack

4.2 System Architecture Layers

The platform is organized across four logical tiers:

Presentation Layer
Browser-based web UI: HTML5, CSS/SCSS, JavaScript
Bootstrap responsive layout framework
jQuery + jQuery UI for DOM manipulation and interactive components
DataTables for server-side paginated tabular data rendering
Highcharts for chart and visualization rendering
TinyMCE 7.0.0 for rich-text document authoring
Selectize.js + Bootstrap Multiselect for enhanced form controls
Font Awesome icon library
Application Layer
PHP 8.2 on Apache HTTP Server
Domain modules: <code>management/</code> , <code>compliance/</code> , <code>governance/</code> , <code>assets/</code> , <code>reports/</code> , <code>assessments/</code> , <code>admin/</code>
Shared procedural service layer: <code>includes/functions.php</code> , <code>includes/authenticate.php</code> , <code>includes/api.php</code>
Action-dispatch pattern for API endpoints in <code>api/index.php</code>
RBAC enforcement via <code>add_session_check()</code> , <code>check_permission()</code> , <code>enforce_permission()</code>

CSRF protection (csrf_magic), HTML sanitization (HTMLPurifier), output escaping (Laminas Escaper)
Plugin-style extras model via *_extra() feature toggle functions
PHPMailer v6.9.1 for SMTP email delivery

Data Layer
MySQL 8.0 (InnoDB) relational database — approximately 77 tables
Direct SQL via mysqli/PDO — no ORM layer
Database-backed PHP session storage for horizontal scalability
Database-backed file/blob storage (files table with content column)
Application-level settings caching via get_setting(..., cached=true)

Infrastructure & Integration Layer
Docker Compose orchestration: web (PHP/Apache) + db (MySQL 8.0) containers
OS Cron for background job scheduling: backup, vulnmgmt, notification, assessments
OAuth integrations: GitHub, Azure / Azure DevOps
SAML SSO capability via SimpleSAMLphp v2.2.0
Jira extra integration for issue linkage
ComplianceForge SCF and UCF framework import connectors
Swagger/OpenAPI documentation for API v2 consumer-facing surface

4.3 Complete Technology Stack

Component	Technology / Version
Frontend Languages	HTML5, CSS/SCSS, JavaScript (ES5+)
UI Frameworks	Bootstrap, jQuery, jQuery UI, EasyUI
Data Visualization	Highcharts (charts), DataTables (tabular)
Rich-Text Editor	TinyMCE 7.0.0
Backend Language	PHP 8.2
Web Server	Apache HTTP Server (mod_rewrite, mod_headers)
Database	MySQL 8.0 / MariaDB-compatible (InnoDB)
ORM	None — direct SQL via mysqli/PDO
Security Libraries	HTMLPurifier 4.16.0, Laminas Escaper 2.13.0, csrf_magic
MFA / 2FA	pragmarx/google2fa v8.0.1 (TOTP)
Email	PHPMailer v6.9.1
Spreadsheet Export	PhpSpreadsheet 1.29.0, OpenSpout v4.22.0
SAML / SSO	SimpleSAMLphp v2.2.0, saml2 v4.6.11



Component	Technology / Version
API Documentation	Swagger-PHP 4.8.7, Swagger-UI v5.12.0
Templating (optional)	Twig v3.9.0
Cron Expression Parsing	dragonmantank/cron-expression v3.3.3
Containerization	Docker Compose (PHP/Apache image + MySQL 8.0)
Dependency Management	Composer (PHP)
Localization	GNU Gettext — multiple language packs

4.4 Database Schema Overview

The RAKSHITAH database (schema defined in rakshitah.sql) comprises approximately 77 tables organized across functional domains. All tables use InnoDB storage engine for full ACID compliance and referential integrity.

Domain	Key Tables
Identity & Access	users, permissions, permission_groups, permission_to_user, sessions, failed_login_attempts, password_reset, roles
Risk Management	risks, pending_risks, risk_scoring, risk_scoring_history, residual_risk_scoring_history, mitigations, mitigation_to_controls, mgmt_reviews, closures, comments, projects
Compliance & Audit	frameworks, framework_controls, framework_control_mappings, framework_control_tests, framework_control_test_audits, framework_control_test_results, framework_control_test_comments
Governance & Docs	documents, document_status, document_exceptions, document_exceptions_status, family, control_class, control_phase, control_priority, control_maturity
Assets	assets, asset_groups, assets_asset_groups, asset_values, risks_to_assets, risks_to_asset_groups
Assessments	assessments, assessment_questions, assessment_answers, assessment_answers_to_assets, assessment_scoring
Platform & Config	settings, audit_log, backups, files, compliance_files, dynamic_saved_selections, fields, file_types, languages, date_formats
Reference/Lookup	category, regulation, location, impact, likelihood, CVSS_scoring, custom_risk_model_values, data_classification, risk_catalog

4.5 Security Architecture

RAKSHITAH implements a defense-in-depth security model spanning authentication, authorization, input validation, data protection, and operational audit trails.

Authentication

- Local username/password authentication with salted, hashed credentials (generateSalt, generateHash)
- Configurable password policy: minimum length, character complexity, minimum/maximum age, reuse limits
- Account lockout after configurable failed login attempts with configurable lockout duration
- Multi-Factor Authentication (MFA/TOTP) via Google Authenticator-compatible implementation, enforceable globally
- OAuth integrations: GitHub OAuth and Azure/Azure DevOps OAuth for SSO-style login flows
- SAML 2.0 SSO capability via SimpleSAMLphp v2.2.0 (via extras/integration path)
- Session-based authentication with optional database-backed session persistence
- Password reset token issuance, validation, and expiry enforcement

Authorization — Role-Based Access Control (RBAC)

- Roles and permissions managed in admin/role_management.php
- Fine-grained permission keys covering: Risk Management, Compliance, Governance, Assets, Project Management, Audit, Administration, API access, Exception approval
- Per-user permission overrides in addition to role-based assignments
- Optional team-based data separation via team_separation_extra() for multi-team isolation
- Admin-only privilege flag separate from RBAC roles for platform administration functions

- Page-level and endpoint-level permission checks on every protected resource

Input Security & Data Protection

- CSRF token protection via csrf_magic on all state-changing forms
- HTML sanitization of rich-text inputs via HTMLPurifier 4.16.0
- Output escaping via Laminas Escaper 2.13.0 to prevent XSS
- Configurable Content Security Policy (CSP) header via admin toggle
- Security headers applied via add_security_headers() on all pages
- File upload policy: configurable allowed extensions, MIME types, and maximum file size
- Application-level encryption for OAuth tokens (GitHub, Azure) before database storage via try_encrypt()/try_decrypt()
- SSL/TLS verification settings for all outbound HTTP connections

Audit Trail

- Comprehensive audit_log table capturing all major platform actions
- write_log() and change localization functions record user, action, timestamp, and before/after state
- Admin audit trail viewer (admin/audit_trail.php) for compliance investigation
- Exception audit log API and test comment/timeline APIs provide granular traceability

5. Data Flows & Process Flows

5.1 User Authentication Flow

The following sequence describes the complete authentication flow from initial login through session establishment:

Step 1	User submits credentials on index.php login page
Step 2	Application loads configuration from includes/config.php and initializes common functions
Step 3	login() in includes/authenticate.php invokes is_valid_user() to verify credentials against stored salt/hash
Step 4	Pre-grant checks: lockout status (failed_login_attempts), password age, required password change flag
Step 5	If MFA is enabled/required: redirect to TOTP verification step. Access withheld until code validated
Step 6	set_user_permissions() loads all RBAC permissions into the PHP session
Step 7	If DB session mode: session written to sessions table via custom sess_write handler
Step 8	grant_access() finalizes authenticated session. add_session_check() protects all subsequent pages
Step 9	On logout: session destroyed, optional DB session record purged

5.2 Core Risk Management Lifecycle Flow

- User opens management/index.php → submits new risk via submit_risk(). Captures: subject, reference ID, regulation/control mapping, source, category, location, team, technology, owner, manager, assessment notes, stakeholder and project links
- User selects scoring methodology (Classic / CVSS / DREAD / OWASP / Custom / Contributing). submit_risk_scoring() writes scoring to risk_scoring and populates risk_scoring_history
- Risk appears in open risk dashboards and management queues with calculated risk score
- Mitigation created via submit_mitigation(). Controls linked via save_mitigation_controls(). Mitigation effort, team, and cost tracked
- Management review performed via submit_management_review(): status, review text, next step, reviewer, comments, and next review date captured. Residual risk recalculated
- Supporting documents, comments, and evidence attachments added at any lifecycle stage
- Risk closed via close_risk() with closure reason, or reopened via reopen_risk() if circumstances change
- All lifecycle transitions written to audit_log for full traceability

5.3 Compliance Management & Audit Flow

- Governance/compliance teams define frameworks and controls in `governance/index.php`. Controls stored in `frameworks` and `framework_controls` tables
- Compliance users define test plans in `compliance/index.php`: testers, stakeholders, frequencies, objectives, test steps, and expected results
- Audit initiation (`compliance/audit_initiation.php`): select controls and frameworks to audit, assign testers, define scope and timing
- Test execution in `compliance/testing.php`: testers record results, attach evidence files, and add timestamped comments
- Results written to `framework_control_test_results` and `framework_control_test_audits` tables
- Failed test result → workflow prompt: user creates a new risk or associates an existing risk for remediation. Risk-test linkage stored in `framework_control_test_results_to_risks`
- Successful test results can trigger closure of linked remediation risks
- Past audit evidence remains queryable for future compliance reporting and regulatory inquiry

5.4 Governance Document Management Flow

- User accesses governance document UI. Documents created via `create_document_api()` with metadata, status, and parent framework linkage
- Rich-text content authored via TinyMCE editor. Document status lifecycle tracked in `document_status` table
- Exception workflow: exception created via `create_document_exception_api()` → reviewed and approved via `approve_exception_api()` → approval recorded in exception audit log
- Parent-child document and framework hierarchies managed through relationship mapping APIs
- Document-to-control and document-to-asset linkages established for cross-module traceability

5.5 Report Generation Flow

- User navigates to a report page under `reports/` and applies filters (teams, owners, frameworks, date ranges, scoring methods)
- PHP reporting functions query MySQL database directly, aggregating risk scores, compliance metrics, and audit statistics
- For the Unified GRC Dashboard: page calls `reports/unified_dashboard_data.php` to retrieve pre-aggregated JSON metrics
- Highcharts renders visualization widgets (pie charts, trend lines, bar charts) in the browser
- DataTables renders paginated, sortable tabular reports with server-side data APIs
- User-specific display settings saved via `dynamic_saved_selections` and `graphical_saved_selections` tables for personalized recurring views
- Reports exportable as spreadsheets via `PhpSpreadsheet` / `OpenSpout`

5.6 Background Job Flows

RAKSHITAH executes four categories of background jobs via OS cron, each corresponding to a PHP script under `cron/`:

Job	Script	Function
Backup	<code>cron/cron_backup.php</code>	Executes configured database/application backup, stores to <code>backups</code> table, enforces <code>backup_remove</code> retention policy
Notification	<code>cron/cron_notification.php</code>	Processes pending notifications (review reminders, overdue risk alerts, assessment deadlines) and sends via <code>PHPMailer</code>
Vulnerability Management	<code>cron/cron_vulnmgmt.php</code>	Polls configured vulnerability data sources, processes findings, and feeds into vulnerability dashboard



Job	Script	Function
Assessments	cron/cron_assessments.php	Processes scheduled assessment sends, reminders, and completion tracking

6. Deployment Architecture

6.1 Deployment Options

RAKSHITAH supports three deployment models to meet varying organizational requirements for data sovereignty, operational control, and infrastructure investment:

Model	Description	Benefits
Cloud-Hosted SaaS	9USRCraft-managed cloud deployment	Fastest time-to-value, zero infrastructure management, automatic updates
On-Premises	Docker Compose on customer infrastructure (LAMP or containerized)	Full data sovereignty, air-gapped deployments, custom security hardening
Hybrid	On-premises application tier with cloud-backed storage or DR	Balance of control and managed services for regulated industries

6.2 Docker Compose Architecture

The reference deployment uses Docker Compose with two primary services:

web	Custom PHP 8.2 + Apache container (docker/php/Dockerfile). Serves the full RAKSHITAH application. Apache mod_rewrite and mod_headers enabled. PHP.ini tuned for GRC workloads.
db	MySQL 8.0 official container. Persistent InnoDB storage. Credentials injected via .env file. All application state, sessions, files, and audit logs persisted here.

Configuration management follows a two-tier approach: Docker .env file for container-level orchestration variables (database credentials, ports), and includes/config.php for application-level constants (DB host, port, install state, session mode). Integration-specific configuration files reside in config/ (e.g., config/azure.php, config/github.php, config/tpdf.php).

6.3 Scalability Considerations

The current architecture is optimized for small-to-medium enterprise deployments. For larger-scale deployments, the following scaling strategies apply:

- Horizontal web tier scaling: multiple PHP/Apache containers behind a load balancer is achievable because sessions are database-backed (not in-memory)
- Database scaling: MySQL primary-replica replication or managed cloud database services (AWS RDS, Azure Database for MySQL) improve read throughput
- File storage: for high-volume deployments, externalizing the files table content to object storage (S3/Azure Blob) reduces database size
- Caching: implementing Redis or Memcached for the settings cache and session store would improve performance at scale
- The modular extras architecture allows selective feature activation to reduce resource consumption in focused deployments

7. Integrations & Extensibility

7.1 Identity & SSO Integrations

Integration	Description	Status
GitHub OAuth	OAuth 2.0 login and account linking via GitHub identity provider	Connected
Azure / Azure DevOps OAuth	OAuth 2.0 SSO via Microsoft Azure AD and Azure DevOps	Connected
SAML 2.0	Enterprise SSO via SimpleSAMLphp v2.2.0 — compatible with Okta, ADFS, Azure AD SAML	Via Extras
Custom Authentication	Pluggable custom_authentication_extra() for LDAP and proprietary IdP integration	Via Extras

7.2 DevOps & Issue Tracking Integrations

- Jira Integration (jira_extra): link RAKSHITAH risks and compliance findings directly to Jira issues for DevSecOps workflow integration
- Azure DevOps OAuth: user authentication and identity federation with Azure DevOps organizations
- GitHub OAuth: developer identity integration enabling GitHub account-linked access to RAKSHITAH

7.3 Compliance Framework Connectors

- ISO 27001: bundled control library (sql/ISO27001.csv) for immediate framework mapping
- ComplianceForge: admin/complianceforge.php and admin/complianceforge_scf.php provide SCF (Secure Controls Framework) integration for enterprise control libraries
- UCF (Unified Compliance Framework): admin/ucf.php provides UCF connector capability for cross-framework harmonization
- Custom framework import: CSV-based framework and control import for proprietary or emerging regulatory frameworks

7.4 API Surface

RAKSHITAH exposes a comprehensive REST-like API for system integration and automation:

- API v1 (api/index.php + includes/api.php): action-dispatch style endpoints covering all major GRC entities — risks, mitigations, reviews, controls, documents, exceptions, assets, audits, comments
- API v2 (api/v2/): formal REST API surface with Swagger/OpenAPI documentation (swagger-php 4.8.7 + swagger-ui v5.12.0)
- JSON response format throughout; supports programmatic consumption by SIEM, ITSM, and BI tools
- API authentication enforced via session or API key mechanisms with RBAC permission checks
- Chatbot/AI integration endpoint (api/chatbot_api.php) for AI-assisted GRC query capabilities

8. Supported Compliance Frameworks

RAKSHITAH ships with pre-built framework templates and control libraries for the industry's most commonly required compliance standards. The platform's flexible framework architecture supports simultaneous management of multiple frameworks with cross-mapping capabilities.

Framework	Description	Availability
ISO 27001:2022	Information Security Management System	Bundled (CSV import ready)
SOC 2 Type I/II	Service Organization Control — Trust Service Criteria	Pre-built template
GDPR	EU General Data Protection Regulation	Pre-built template
HIPAA	Health Insurance Portability and Accountability Act	Pre-built template
PCI-DSS	Payment Card Industry Data Security Standard	Pre-built template
NIST CSF	NIST Cybersecurity Framework	Custom import
ComplianceForge SCF	Secure Controls Framework — 1,000+ controls	SCF connector
UCF	Unified Compliance Framework — cross-framework harmonization	UCF connector
Custom Frameworks	Organization-specific or emerging regulatory requirements	CSV import

9. Application Structure & Codebase Organization

RAKSHITAH is organized as a single application repository with domain-aligned directories, shared utility libraries, and a clear separation between presentation, business logic, and data concerns.

Directory	Purpose
management/	Core risk workflow UI — risk submission, scoring, mitigation, reviews
compliance/	Compliance tests, audit initiation, active/past audits
governance/	Frameworks, governance documents, exceptions
assets/	Asset inventory, asset groups, asset-risk mapping
assessments/	Assessment questionnaires, external participant access
incidents/	Incident tracking and management
vulnerabilities/	Vulnerability dashboard and cron-fed processing
reports/	Reporting pages, unified dashboard, JSON data endpoints
admin/	Platform administration: users, roles, mail, backup, security, integrations
api/	API v1 endpoints and chatbot endpoint
api/v2/	API v2 with OpenAPI/Swagger documentation
includes/	Shared service layer: functions.php, authenticate.php, api.php, cron.php, mfa.php
integrations/	OAuth handlers: GitHub, Azure, TPRF
config/	Integration configuration files (github.php, azure.php, tprf.php)
cron/	Background job scripts: backup, notification, vulnmgmt, assessments
sql/	Database schema (rakshitah.sql), seed data, ISO27001.csv
languages/	Localization packs for multi-language support
docker/	Docker container build artifacts and php.ini
vendor/	Composer-managed PHP dependencies
css/, scss/, js/, images/	Frontend UI assets and compiled stylesheets

10. Conclusion

RAKSHITAH represents a comprehensive, production-grade GRC platform that delivers enterprise-class governance, risk, and compliance capabilities with the flexibility, customizability, and cost-effectiveness that modern organizations require.

By unifying Risk Management, Compliance Management, Audit Management, Governance & Document Management, Asset Management, Assessments, Incident Management, Vulnerability Management, and Executive Reporting within a single, cohesive platform, RAKSHITAH eliminates the fragmented tooling that plagues most organizational GRC programs.

The platform's modular monolith architecture, built on proven PHP/Apache/MySQL infrastructure with Docker-based deployment, ensures operational reliability, broad hosting compatibility, and straightforward administration. The layered security architecture — spanning RBAC, MFA, CSRF protection, HTML sanitization, encrypted credential storage, and comprehensive audit trails — provides the security assurances that regulated industries demand.

RAKSHITAH's extensibility through its extras model, REST API, and integration connectors for Jira, Azure DevOps, GitHub, SAML SSO, and compliance framework libraries ensures that the platform grows with organizational GRC maturity rather than constraining it.

In essence, RAKSHITAH is not just a GRC tool — it is your enterprise's guardian, ensuring that risks are managed, vulnerabilities are minimized, compliance is continuously upheld, and governance is embedded into the operational fabric of the organization.

To learn more or request a demo:

sales@9usrcraft.com | +91 992-036-0009 | www.9usrcraft.com



© 2025 9USRCraft LLP. All rights reserved. RAKSHITAH is a trademark of 9USRCraft LLP.

This document is confidential and intended solely for the recipient. Technical specifications are subject to change without notice.